Electric Drives and Controls | Hydraulics | Linear Motion and Assembly Technologies | Pneumatics | Service

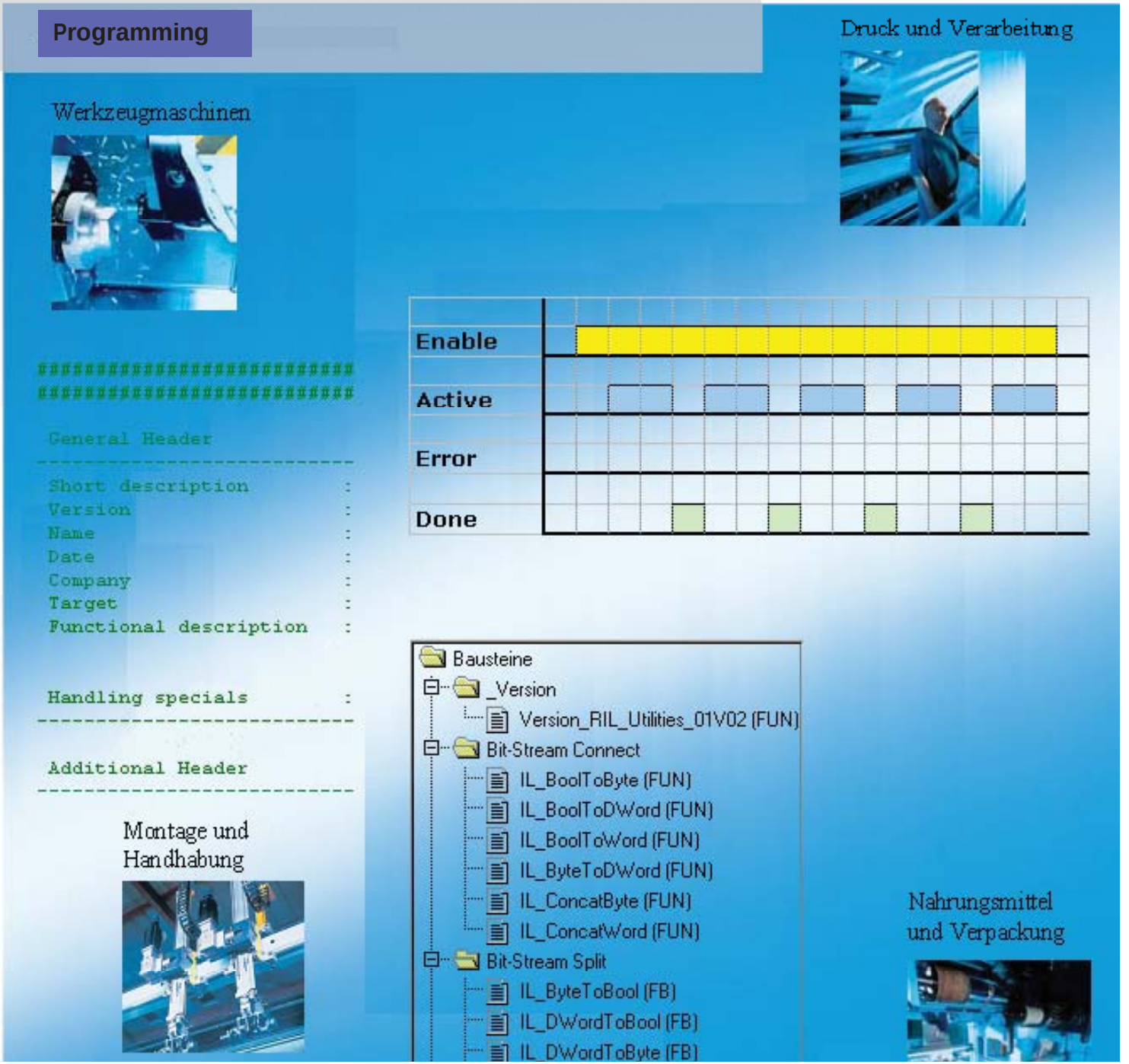
Rexroth
Bosch Group

Exclusively for internal use only!

Rexroth IndraLogic StyleGuide
Guidelines across systems

Edition 2.2

Programming



Title

IndraLogic
Programming guide lines across systems
04VRS

Type of documentation

Programming

Document Type code

Internal File Reference

Purpose of Documentation?

This documentation serves as internal programming guideline.

Record of Revisions

Description	Release Date	Notes
IndraLogic Guidelines across systems	01	
IndraLogic Guidelines across systems	02	
IndraLogic Guidelines across systems	03	02V01
IndraLogic Guidelines across systems	04	02V02

Validity

The specified data is for product description purposes only and may not be deemed to be guaranteed unless expressly confirmed in the contract. All rights are reserved with respect to the content of this documentation and the availability of the product.

Published by

Bosch Rexroth AG
Bgm.-Dr.-Nebel-Str. 2 • D-97816 Lohr a. Main

Telephone +49 (0)93 52/40-0 • Tx 68 94 21 • Fax +49 (0)93 52/40-48 85

http://www.boschrexroth.com/

Working party for functions and function blocks across systems

Note

This document has been printed on chlorine-free bleached paper.

1 Contents

2 General information

The goal of this documentation to unify all functions, function blocks and data blocks from BRC across systems based on the PLCopen concerning the appearance and the behavior. This enables the user for eas of use and consistency across the systems (MLC, MTX, Drive, etc. ...).

The PLCopen standard is the basis for the application. The specific implementation directions are described under the heading "Definition of use".

The functions and function blocks provided by BRC are identified by the defined token from Bosch Rexroth (see Fig. 6-2).

3 Standardization header

Definition of use

Note: The use of the header template is mandatory for all BRC functions and function blocks!

Motivation

A header displays important information about the allocated program code quick and clear.

Description

The Header contains the following information in the specified sequence:

- Brief description with purpose of use
- Version status
- Name and department of the originator
- Revision date
- Company
- Target
- Detailed description of the functionality (optional)
- Supplementary and further information (e.g., parameterization notes, etc..)

Note: The header must be created in **English**.

Name of target	System token	Branch
MTX_PNC_P	MTX	machine tools
IndraMotion MTX CMP60	MTX	machine tools
IndraMotion MLC L40 01VRS	MLC	all except of machine tools
IndraDrive MPH02	MLD-S	all branch groups
IndraDrive MP03	MLD-S	all branch groups
SYNAX200-MotionLogic 12VRS (PPC-P)	SYNAX	printing and converting machines
SYNAX200-MotionLogic 12VRS (PPC-R)	SYNAX	printing and converting machines

Fig. 3-1: Examples for Targets (target systems)

Fig. 5-1 documents the use of the header in IndraLogic with an example.

Implementation

To include the header the following template should be used. For this purpose mark the whole contents of the text file and copy it to the clipboard.

Declaration "Normal" Then insert the content of the clipboard in the declaration part of IndraLogic before all other code.

Declaration as table When using the tabular view, insert the content of the clipboard at tab "Info".

Font formatting The standard font formatting of IndraLogic is "Courier New, Standard, 8". This is a non-proportional font, also named as Monospace font. It has a fixed character pitch.

The use of this font is mandatory for the programming with IndraLogic!

Header template

"Header Vorlage.txt"

4 Standardization history

Definition of use

Note: The use of the history template is mandatory for all BRC functions and function blocks!

Motivation

To document the development of the code, it is useful to add a record of revisions. A standardization takes place to get a consistent style.

Description

The history contains the following information:

- Version status (e.g., 01V01)
- Name or token and department of the software engineer (e.g., BRC-ESP1 (EH))
- Release date (e.g., 2005-01-27)
- Description of the changes

Note: The history must be created in **English**.

Fig. 5-1 documents the use of the history in IndraLogic with an example.

Implementation

To include the history the following template should be used. For this purpose mark the whole contents of the text file and copy it to the clipboard.

Declaration "Normal"

It is advisable to add the history at the end of the declaration (see Fig. 5-1).

Note: If the history is added at the end of the declaration, a change of the declaration to the tabular view together with a modification leads to a complete loss of the history!

Declaration as table

When using the tabular view, the history must be insert at tab "Info" directly after the header.

Font formatting

The standard font formatting of IndraLogic is "Courier New, Standard, 8". This is a non-proportional font, also named as Monospace font. It has a fixed character pitch.

The use of this font is mandatory for the programming with IndraLogic!

History template

"History Vorlage.txt"

5 Implementation example header and history

```
(*#########################################################################################################*)
(*#########################################################################################################*)

(* General Header *)
(*-------------------------------------------------------------------------------------------------------*)
(* Shortdescription       : This function block provide the communication between device xyz *)
(*                          via Profibus and the programmable logic controller *)
(* Version                : 1.3 *)
(* Name                   : Max Mustermann *)
(* Date                   : 2004-02-02 *)
(* Company                : Bosch Rexroth AG *)
(* Target                 : SYNAX200-MotionLogic; VisualMotion *)
(* Functional description : A communication with device xyz is only possible over a special *)
(*                          multiplex process. The function block decodes and encodes the *)
(*                          telegram and makes sure that ... *)
(* Handling specials      : Attend the following points: *)
(*                          - It's essential that the device xyz is connected with the *)
(*                            control unit. *)
(*                          - Connect the device to the Profibus and provide that the bus is *)
(*                            running without any error. *)
(*-------------------------------------------------------------------------------------------------------*)

(* Additional Header *)
(*-------------------------------------------------------------------------------------------------------*)
(* Customer               : Koenig & Bauer AG *)
(* Machine                : FA0815STX *)
(*-------------------------------------------------------------------------------------------------------*)

(*#########################################################################################################*)
(*#########################################################################################################*)
PROGRAM ZZMainProgram
VAR
    stComData: ComData;
    enDiagData: DiagData;
    fbGetJogMode: MT_GetJogMode;
    arStateInfo: ARRAY[0..10] OF INT;
END_VAR
(*#########################################################################################################*)

(*Modification - History*)
(*-------------------------------------------------------------------------------------------------------*)
(* Version     : 1.1*)
(* Name        : Hugo Muster *)
(* Date        : 2003-08-22 *)
(* Comment     : Error correction in modul ... *)
(**)
(* Version     : 1.2 *)
(* Name        : Max Mustermann *)
(* Date        : 2003-12-15 *)
(* Comment     : Functional add on ... *)
(**)
(* Version     : 1.3 *)
(* Name        : Max Mustermann *)
(* Date        : 2004-02-02*)
(* Comment     : Bug fixing: *)
(*               - communication problem when the profibus ... *)
(*               - decode error by .... *)
(*-------------------------------------------------------------------------------------------------------*)

(*#########################################################################################################*)
```

HeaderExample_EN.bmp

Fig. 5-1: Example header/history (declaration "Normal")

6 Standardization type identifier

Definition of use

Note: The use of standardized type identifier is mandatory for all BRC functions and function blocks!

Motivation

A consistent procedure at the allocation of the names for types is helpful to radical increase the readability of the program code. So the practice of third persons is facilitated considerably and the possible troubleshooting is shortened.

Description

In principle it can be differed between two types:

Type1
- Types that can contain program code, including programs, function blocks and functions.

Type2
- Types that don´t contain program code, including structures, arrays and enumerators, as well as all IEC data types as e.g., String, Integer or Real.

According to PLCopen all names for Type1 are formed considering the upper and lower case. All names for Type2 are only defined with upper case.

Standardization of type identifiers for programs, functions and function blocks (type 1)

According to PLCopen type identifiers for programs, functions and function blocks are formed considering the upper and lower case.

Note: There's a conflict between PLCopen and IEC standard. Functions and function blocks according to IEC standard are capitalized (e.g., R_TRIG, TON, etc.). At functions and function blocks of PLCopen the upper and lower case must be considered (e.g., MC_MoveAbsolute).

The programming guideline tries to be geared to the PLCopen standard.

Programs Type identifiers are formed without underlines, prefix or suffix. Upper and lower case must be considered. The type identifier should be in English.

Example:

Fig. 6-1: Types: programs

Functions and function blocks

Token are defined for system specific functions and function blocks as well as for system independent functions and function blocks of Bosch Rexroth. These token must be added to the type identifier as prefix with underline. Suffixes as well as further underlines are not allowed. The upper and lower case must be considered. The type identifier should be in English.

Note: Function blocks and function that are characterized with "IL", "MC", "MB" or "IH" are not unavoidable provided or supported by any system (e.g., MTX, MLC, etc.). Every system that provides a module with "IL", "MC", "MB" or "IH" in the name, must ensure that its behavior to outwards is system overlapping the same. That means that e.g., the function block "IL_SercosAttribute" must be the same to outwards in every system that makes it available concerning the style and the behavior.

For this reason functions and function blocks that are characterized with "IL", "MC", "MB" or "IH" must be authorized by the Bosch Rexroth "Working party for functions and function blocks across systems"!

Prefix	System dependence	Allocation	Description	Example
MC	no	PLCopen	100% PLCopen module	MC_MoveAbsolute
MB	no	Motion oriented according to PLCopen	modules with motion oriented functionalities that are not PLCopen certificated, but they are geared to this standard	MB_WriteParameter
IL	no	-	all system independent functions and function blocks that are not PLCopen and motion oriented	IL_SercosAttribute
IH	no	Hardware	Functions and function blocks without system reference (e.g., MLC, MTX, ...) that only run at the corresponding hardware (e.g., CML40, CMP60, ...)	IH_SetDisplay
ML	yes	MLC	MLC specific functions and function blocks	ML_WriteCyclicPosition
MP	yes	IndraLogic	IndraLogic specific functions and function blocks	
MS	yes	Synax	Synax specific functions and function blocks	MS_ReadSingleParameter
MSV	yes	Synax + VisualMotion	Functions and function blocks for the systems Synax and VisualMotion	MSV_ReadMaxValue
MT	yes	MTX	MTX specific functions and function blocks	MT_NcBlk
MV	yes	VisualMotion	VisualMotion specific functions and function blocks	MV_Hysteresis
MX	yes	Drive PLC	Functions and function blocks especially for drive PLC	MX_SetDeviceMode

Fig. 6-2: Overview prefixes for functions and function blocks

Example:

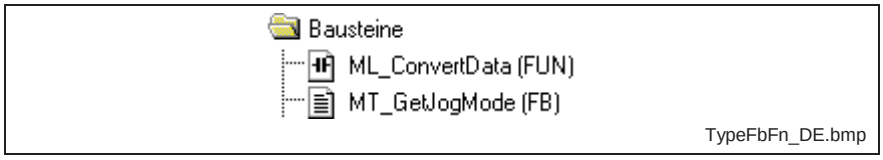


Fig. 6-3: Types: Function block & functions

Standardization of type identifiers for structures, arrays and enumerators (Type2)

According to PLCopen type identifiers for structures, arrays and enumerators (Type2) are only formed with upper case.

Note: If structures, arrays or enumerators are bound to a special system, the name should contain the prefixes that are defined in Fig. 6-4.

Structures

Type identifiers for structures are only formed with upper case. To improve the readability underlines can be used. The type identifier should be in English.

Example:

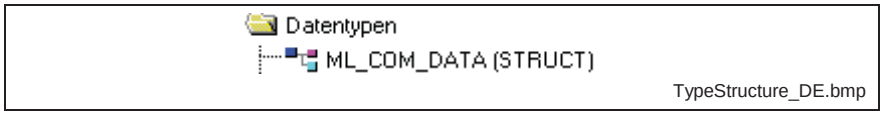


Fig. 6-4: Types: Structures

Arrays

Type identifiers for arrays are only formed with upper case. To improve the readability underlines can be used. The type identifier should be in English.

Example:

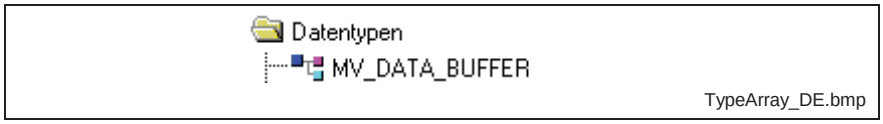


Fig. 6-5: Types: arrays

Note: If arrays are directly defined in the declaration, the definition of a type identifier is invalid.

Enumerators

Type identifiers for enumerators are only formed with upper case. To improve the readability underlines can be used. The type identifier should be in English.

Example:

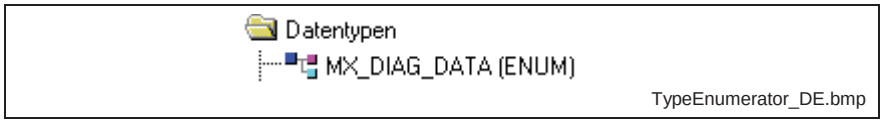


Fig. 6-6: Types: Enumerators

7 Standardization variable identifier and sequence identifier

Definition of use

Note: The use of standardized variables and sequence identifier is mandatory for all BRC functions and function blocks!

Note: ***Limitations:***

The use of prefixes is optional and is not obligatory for the creation of standardized variable and sequence identifiers. If prefixes should be used, the following defined prefixes must be used for all variables!

To be PLCopen conformable nevertheless, it is recommended that interfaces at functions and function blocks are not provided with prefixes. But within functions and function blocks prefix afflicted identifiers should be used.

Motivation

A consistent procedure at the allocation of the names for variables and sequence elements is helpful to radical increase the readability of the program code. So the practice of third persons is facilitated considerably and the possible troubleshooting is shortened.

Description

Standardization of the identifiers of sequences

Sequences are built of steps, actions and transitions. The following points must be considered when building the identifiers for the sequence elements.

Check list

- Consider upper and lower case
- Make instance identifier in English
- Add prefixes to the name in lower case and without underline

Sequence element	Prefix	Example
Step	ste	steDriveControl
Action	act	actActivateDrive
Transition	tra	traDriveActive

Fig. 7-1: Identifiers of sequences

Standardization of the identifier of complex data types

Instances can be build of complex data types as function blocks, structures, arrays and enumerators. The following points must be considered when building instance identifiers of complex data types.

Check list

- Consider upper and lower case
- Make instance identifier in English
- Add prefixes to the name in lower case and without underline

Data type	Prefix	Instance example	Type example
Function block	fb	fbJogAxisX1	MT_Jogging
Structure	st	stDeviceCommunication	MX_COM_DATA
Array	ar	arStateControlUnit	MV_STATE_INFO
Enumerator	en	enDeviceDiagnosis	ML_DIAG_DATA

Fig. 7-2: Instance identifier of complex data types

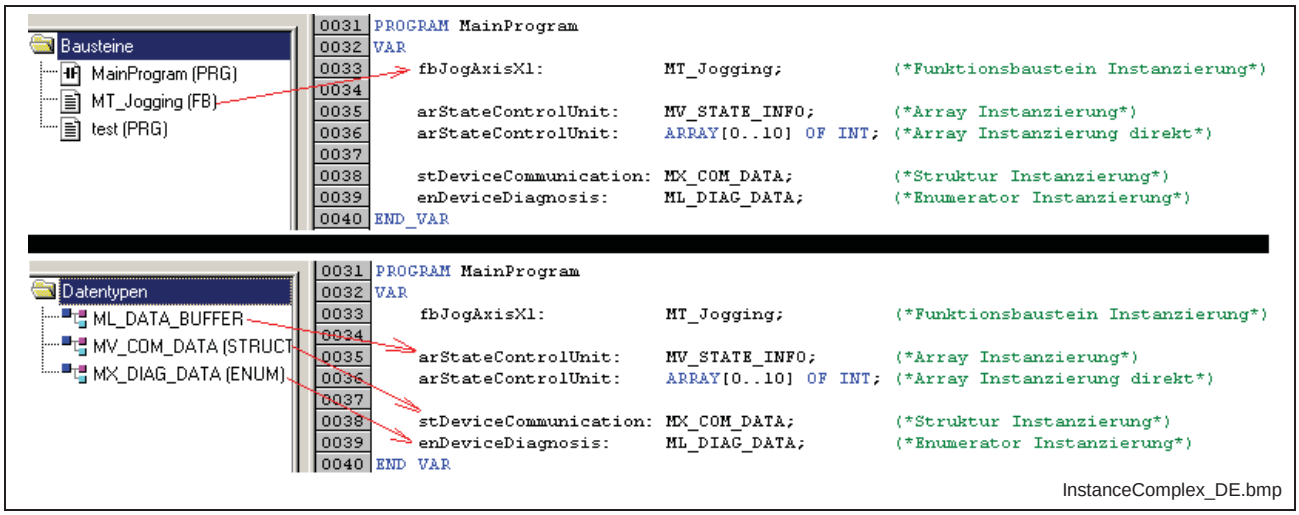


Fig. 7-3: Example instance of complex data types

Standardization of the identifier of simple data types

The following prefixes should be used for corresponding instance names of simple data types. The following points must be considered when building the instance identifier of simple data types.

Check list

- Consider upper and lower case
- Make instance identifier in English
- Add prefixes to the name in lower case and without underline

Data type	Prefix	Example	Memory allocation	Data type designation	Data type description
BOOL	b	bVar	1 Bit	Boole	Bit oriented boolean format
BYTE	by	byVar	8 Bit	Byte	Bit oriented Byte format
WORD	w	wVar	16 Bit	Word	Bit oriented format with simple word length
DWORD	dw	dwVar	32 Bit	Double Word	Bit oriented format with double word length
LWORD	lw	lwVar	64 Bit	Long Word	Bit oriented format with fourfold word length
SINT	si	siVar	8 Bit	Short Integer	integer signed format with shortened length
INT	i	iVar	16 Bit	Integer	integer signed format with simple length
DINT	di	diVar	32 Bit	Double Integer	integer signed format with double length
LINT	li	liVar	64 Bit	Long Integer	integer signed format with fourfold length
USINT	usi	usiVar	8 Bit	UnsignedShort Integer	integer unsigned format with shortened length
UINT	ui	uiVar	16 Bit	Unsigned Integer	integer unsigned format with simple length
UDINT	udi	udiVar	32 Bit	Unsigned Double Integer	integer unsigned format with double length
ULINT	uli	uliVar	64 Bit	Unsigned Long Integer	integer unsigned format with fourfold length
REAL	r	rVar	32 Bit	Real	real number with simple length

LREAL	lr	lrVar	64 Bit	Long Real	real number with double length
STRING	str	strVar	8 Bit per character	String	string of 1-255 characters (ANSI code possible)
WSTRING	wstr	wstrVar	16 Bit per character	Wide String	string of 1-65535 characters (UNI code possible)
TIME	t	tVar	32 Bit	Time	Time format
DATE	d	dVar	32 Bit	Date	Date format
TIME_OF_DAY	tod	todVar	32 Bit	Time Of Day	Time of day format
DATE_AND_TIME	dat	datVar	32 Bit	Date And Time	Date and time format
POINTER TO ???	p???	p???Var	32 Bit	Pointer To *???*	Pointer / address of a variable with special data type
POINTER TO DWORD	pdw	pdwVar	32 Bit	Pointer To Double Word	Example: Pointer to a double word variable

Note: The data types ***LWORD***, ***LINT***, ***ULINT*** and ***WSTRING*** are not supported by IndraLogic at the moment
???: Data type of the variable, the pointer points to

Fig. 7-4: Identifiers of simple data types

Suffixes for identifiers

The instance name of any data types can be provided with suffixes for source identification purposes in addition. This suffixes must be added to name with an underline. The following table of suffixes can be extended, if necessary.

Suffix	Meaning	Example
i	hardware input	bAxisTravelLimit_i
q	hardware output	bLockDoor_q
gb	global variable	stControlState_gb
m	local marker	iComData_m

Fig. 7-5: Examples for suffixes of simple data types

Standardization of the identifiers of constants

Constants give defined states a fixed name, that means they give a value a named meaning in the code. If constants are copied to other variables, an information about the data type makes sense. To distinguish constants from all other variables, the name is formed exclusively with upper case. These names can contain underlines for better readability. Only the data type designation should be prefixed to the identifier in lower case and without underline. The identifier should be in English.

Check list

- Form identifier exclusively in upper case with underlines
- Precede prefixes to the name in lower case and without underline
- Make instance identifier in English

The following table shows some examples.

Constant name	State	Data type	Declaration example
rDRV_POSITION_REACHED	drive has reached the defined position	REAL	rDRV_POSITION_REACHED : REAL :=123.45;
bDOOR_CLOSED	guard door is closed	BOOL	bDOOR_CLOSED : BOOL := TRUE;
usiTURRET_POSITION_5	turret is at position 5	USINT	usiTURRET_POSITION_5 : USINT := 5;

Fig. 7-6: Examples constant identifiers

8 Definition of standard interfaces at function blocks

General information

The definition of standard interfaces is the most important and the most difficult part of this guideline. The functions to program are often complex and can hardly be adjusted to a predefined schema. The challenge for the software engineers is to realize the requested functionality and to present the same behavior at the standardized interfaces to the customer.

Definition of use

Note: The use of the standard interfaces for all system-overlapping function blocks with the token "IL", "IH" "MC" and "MB" (see Fig. 6-2) is stringent described.

Only at "MC" modules differences are allowed in consequence of the guidelines of PLC-Open.

Note: The standardized interfaces are optional for system-specific function blocks (MT, MX, ML, ..), but they are emphatic recommended!

If the standardized identifiers are used, their behavior must comply with the described one. The identifiers may not be used in connection with another behavior in no case.

If interfaces with another behavior are required, they must get other identifiers as the standardized one that are described here!

Motivation

Most of the function blocks have a input for activation and an output that displays the processing without error. Often an additional output is necessary that identifies the processing time. For the definition of errors further outputs are defined that are described in detail in section "9 Standardized error handling".

A standardized naming as well as the same behavior of these standard interfaces increases the comprehension, shortens the practice and relieves the support.

Description

Module type

With function blocks it is possible to encapsulate complex tasks reusable and to activate them via defined interfaces. The processing can be state- or edge-controlled. It is differentiated between tasks that can be handled terminating and tasks that require permanent access after activated once.

State-controlled If a function block repeats its task always at a defined status after switching on, it works "state-controlled".

Edge-controlled If the function block fulfills a task exactly once after switching on, it works "edge-controlled".

Terminating processing If a module can process its task finally and terminates its work, it is a "Terminating processing".

Permanent processing If a module can not finally terminate its task, but it is durable in access, it is defined as "Permanent processing".

The following four module types result from the combination:

Control	Processing	Example
state-controlled (Enable)	terminating (Done)	Durable reading of a parameter (if reading is completed, the module has terminated its task. It repeats its task as long as the control input (Enable) remains set.) e.g., MB_ReadParameter
state-controlled (Enable)	permanent (In….)	Control can be terminated (If the control is active, it works as long as the control input (Enable) is reset and deactivates it.)
edge-controlled (Execute)	terminating (Done)	Once writing of a parameter (If writing is completed, the module has terminated its task. A edge at the control input (Execute) is necessary again to start rewriting.) e.g., MB_WriteParameter
edge-controlled (Execute)	permanent (In….)	Control that can not be terminated (If the control is once started via the control input (Execute), it can not be deactivated via the interfaces of this module any more.) e.g., MC_MoveVelocity

Fig. 8-1: Overview module types

Interfaces that identify the module type

Inputs

To mark, if a function block is state- or edge-controlled, two different variable names for the inputs are used for the activation of the function block.

- Enable = state-controlled
- Execute = edge-controlled

Outputs

To mark, if a function block works terminating or permanent, different variable names are used for the outputs that identify the processing status.

- Done = terminating processing
- In….. = permanent processing (e.g., InOperation, InSync, …)

The optional output Active can be used at terminating and permanent working modules, but the behavior is different in every case!

All outputs for the error identification (Error, ErrorID, ErrorIdent) can be found at terminating and permanent working modules and they both have the same behavior.

Overview of standardized identifiers

Identifier (without data type prefix)	**Description**
Inputs	
Enable	Activation input of state-controlled FBs
Execute	Activation input of edge-controlled FBs. Other FBs can disturb it.
ExecuteLock	Activation input of edge-controlled FBs. As long as this input is TRUE, the FB can not be disturbed by other.
Outputs	
Active	Output that identifies the processing time
Done	Output processing terminated successfully and the data outputs are valid.
In……	Output signals that the module works at its task and the data outputs are valid.
CommandAborted	This output displays that the FB was aborted (e.g., by another FB).
Error	Output processing terminated with error
ErrorID	Output for rough error classification
ErrorIdent	Output for detailed error classification

Fig. 8-2: Overview standardized identifiers

Inputs and outputs of state-controlled function blocks with terminating processing

Control / Processing	I/O	Variable name	Description
state-controlled / terminating	I	Enable	With a positive edge of Enable the input variables are included. New input values get effective not until a fresh positive edge of Enable. "Enable" must be TRUE during the whole module processing! The clearing of "Enable" aborts the processing and sets the inputs "Done", "Active", "CommandAborted" and "Error" to FALSE.
	O	Done	If "Done" is TRUE, the function block has terminated its task successfully and is in final condition. Data outputs are valid now. "Active", "Error" and "CommandAborted" are FALSE! "Done" remains TRUE for exactly one PLC cylce. If Enable is still TRUE afterwards, the module starts its processing again.
	O	Active	If "Active" is TRUE, the function block works its real task and is in intermediate result. Potential pre-processings are not identified with this output! "Done", "Error" and "CommandAborted" are FALSE!
	O	CommandAborded	If "CommandAborted" is TRUE, the function block was aborted and is in a final condition. "Done", "Active" and "Error" are FALSE! "CommandAborted" remains TRUE till the control input "Enable" is cleared. So an additional reset input is not necessary.
	O	Error	If "Error" is TRUE, the function block was aborted because of an error and is in a final condition. "Error" remains TRUE till the control input "Enable" is cleared. So an additional reset input is not necessary.

Fig. 8-3: I/Os of state-controlled FBs with terminating processing

Note: If it is necessary that special inputs are not only assumed with the edge at "Enable", but cyclically during module processing, it must be explicit documented and not identified with variable names!

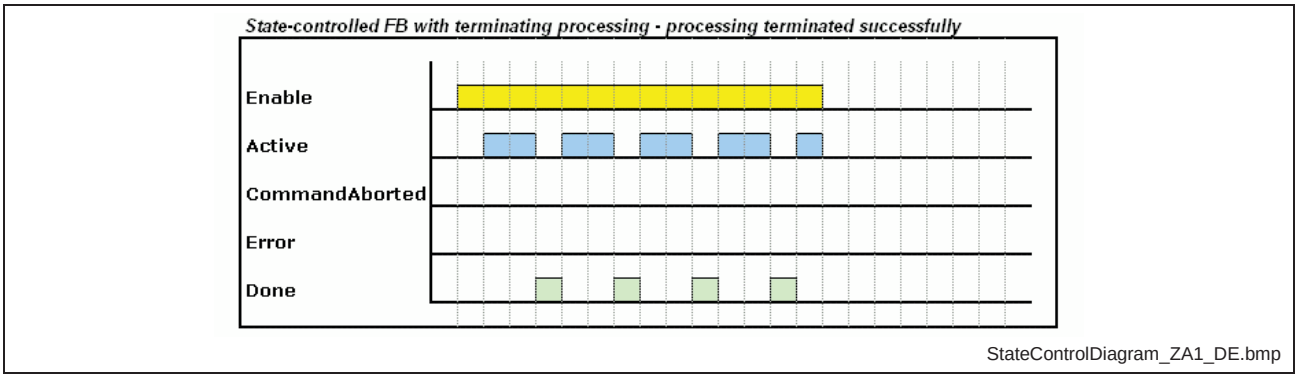
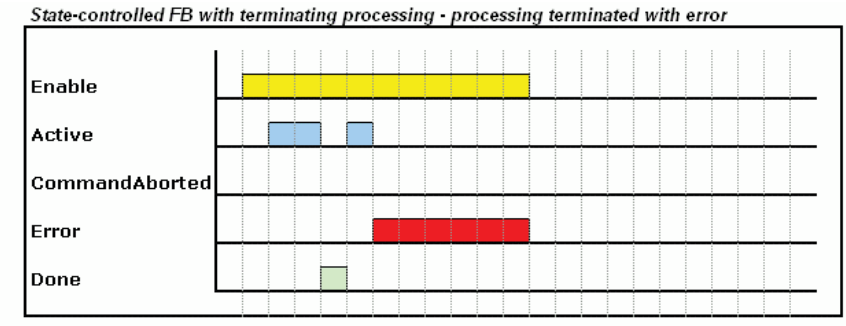
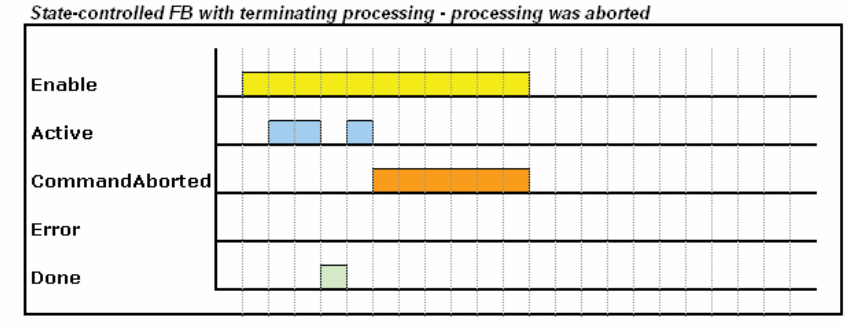


Fig. 8-4: I/Os of state-controlled FBs with terminating processing – processing terminated successfully



StateControlDiagram_ZA2_EN.bmp

Fig. 8-5: I/Os of state-controlled FBs with terminating processing – processing terminated with error



StateControlDiagram_ZA3_EN.bmp

Fig. 8-6: I/Os of state-controlled FBs with terminating processing – processing was aborted

Inputs and outputs of edge-controlled function blocks with terminating processing

Control / Processing	I/O	Variable name	Description
edge-controlled / terminating	I	Execute	With a positive edge of Execute the input variables are included. New input values get effective not until a fresh positive edge of Execute. The change of the edge at "Execute" is sufficient to start the module. For further processing the status of "Execute" is not relevant. With a fresh change of edge during processing the old task is discarded, the inputs are reassumed and the task is continued with the new values (re-trigger).
	O	Done	If "Done" is TRUE, the function block has terminated its task successfully and is in a final condition. Now data outputs are valid. "Active", "Error" and "CommandAborted" are FALSE! If the control input "Execute" is FALSE at task termination, "Done" remains TRUE for exactly one PLC cycle. If "Execute" is TRUE, "Done" remains TRUE till "Execute" is cleared.
	O	Active	If "Active" is TRUE, the function block works its real work and is in intermediate result. Potential pre-processings are not identified with this output! "Done", "Error" and "CommandAborted" are FALSE!

	O	CommandAborded	If "CommandAborted" is TRUE, the function block was aborted and is in a final condition. "Done", "Active" and "Error" are FALSE! If the control input "Execute" is FALSE at the time of abortion, "CommandAborted" remains TRUE for exactly one PLC cycle. If "Execute" is TRUE, "CommandAborted" remains TRUE till "Execute" is cleared.
	O	Error	If "Error" is TRUE, the function block was aborted because of an error and is in a final condition. If the control input "Execute" is FALSE at occurring error, "Error" remains TRUE for exactly one PLC cycle. If "Execute" is TRUE, "Error" remains TRUE till "Execute" is cleared.

Fig. 8-7: I/O of edge-controlled FBs with terminating processing

Note: If it is necessary that special inputs are not only assumed with the edge at "Execute", but cyclically during module processing, it must be explicit documented and not identified with variable names!

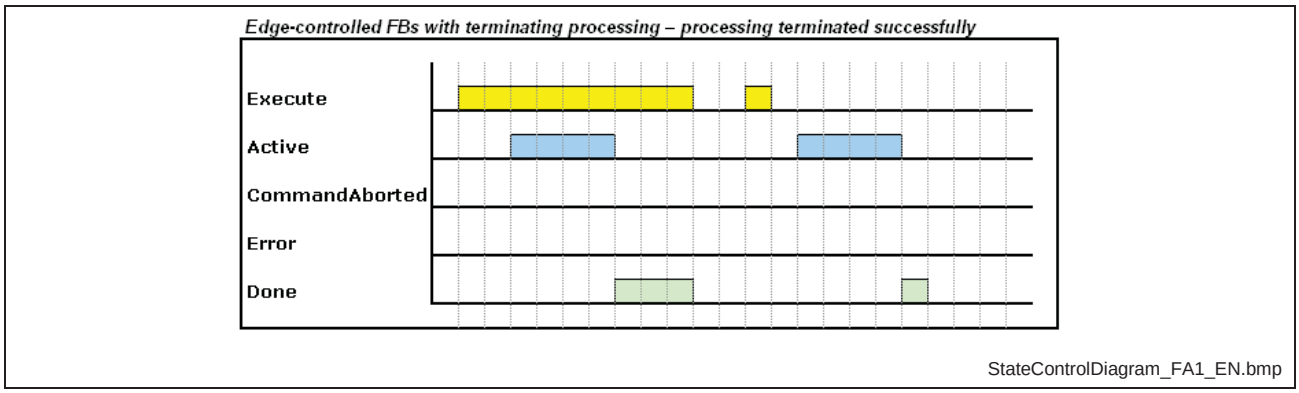


Fig. 8-8: I/Os of edge-controlled FBs with terminating processing – processing terminated successfully

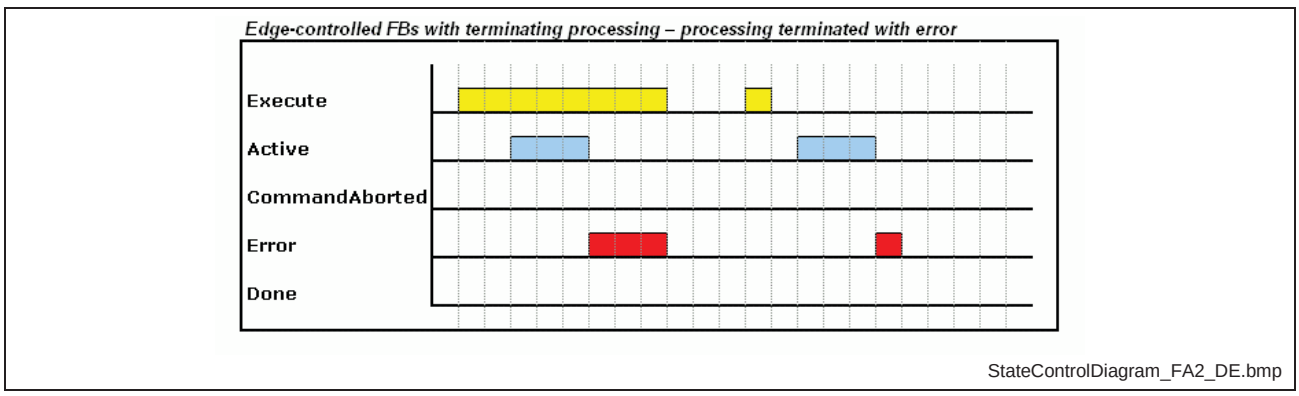


Fig. 8-9: I/Os of edge-controlled FBs with terminating processing – processing terminated with error

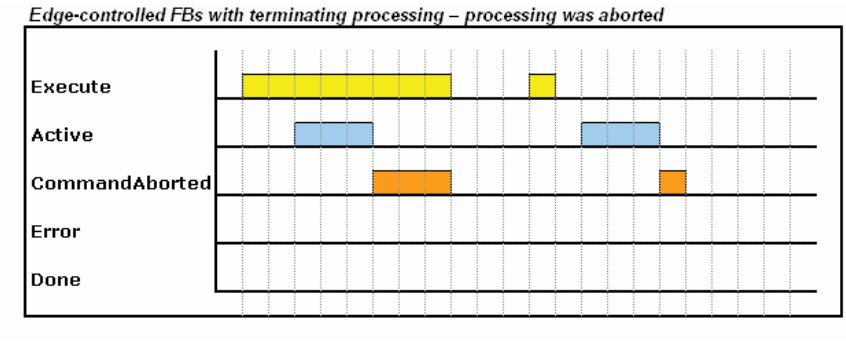


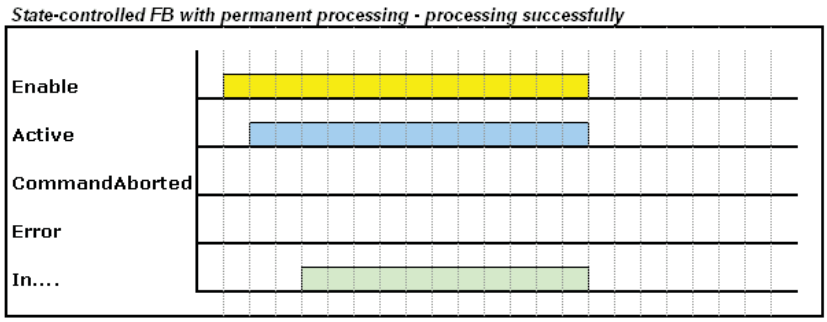
Fig. 8-10: I/Os of edge-controlled FBs with terminating processing - processing was aborted

Inputs and outputs of state-controlled function blocks with permanent processing

Control / Processing	I/O	Variable name	Description
state-controlled / permanent	I	Enable	With a positive edge of Enable the input variables are included. New input values get effective not until a fresh positive edge of Enable. "Enable" must be TRUE during the whole module processing! If "Enable" is cleared, the processing is aborted and the outputs "In….", "Active", "CommandAborted" and "Error" are set to FALSE.
	O	In….	If "In…." is TRUE, the function block has reached its final position, but is operative further on to "keep" the achieved result and thus in a durable final condition. Data outputs are valid now. As the FB remains operative, "Active" must also remain TRUE. "Error" and "CommandAborted" are FALSE! As long as "Enable" is TRUE, "In…." remains also TRUE.
	O	Active	If "Active" is TRUE, the function block works its real task. Potential pre-processings are not identified with this output! As the FB remains permanent operative, "Active" remains TRUE till the FB is switched off via the control input "Enable" or is terminated with "Error" or "CommandAborted". As long as "Active" is TRUE, "Error" or "CommandAborted" must be FALSE!
	O	CommandAborded	If "CommandAborted" is TRUE, the function block was aborted and is in a final condition. "In….", "Active" and "Error" are FALSE! "CommandAborted" remains TRUE till the control input "Enable" is cleared. So an additional reset input is not necessary.
	O	Error	If "Error" is TRUE, the function block was aborted because of an error and is in a final condition. "Error" remains TRUE till the control input "Enable" is cleared. So an additional reset input is not necessary.

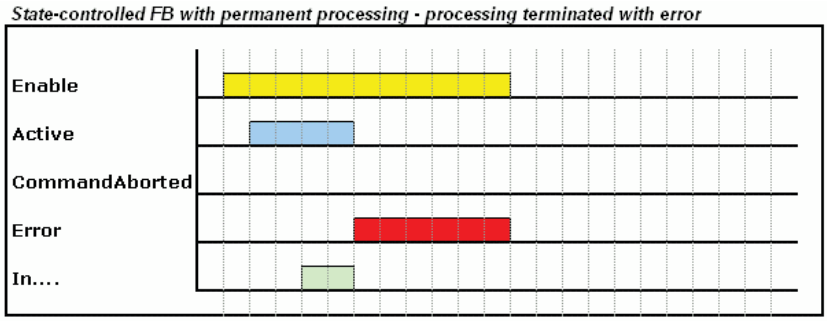
Fig. 8-11: I/O of state-controlled FBs with terminated processing

Note: If it is necessary that special inputs are not only assumed with the edge at "Enable", but cyclically during module processing, it must be explicit documented and not identified with variable names!



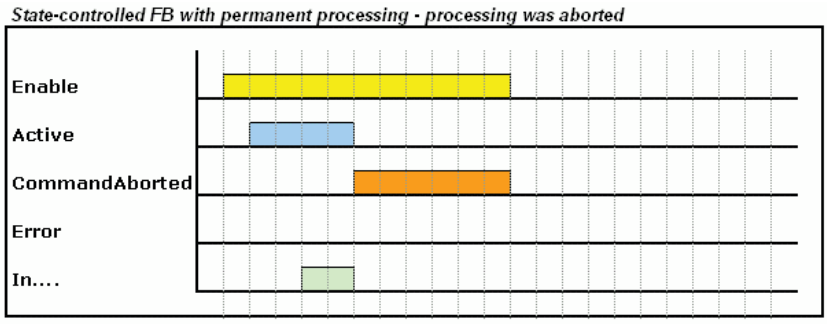
StateControlDiagram_ZP1_EN.bmp

Fig. 8-12: I/Os of state-controlled FBs with permanent processing - processing successfully



StateControlDiagram_ZP2_EN.bmp

Fig. 8-13: I/Os of state-controlled FBs with permanent processing - processing aborted with error



StateControlDiagram_ZP3_EN.bmp

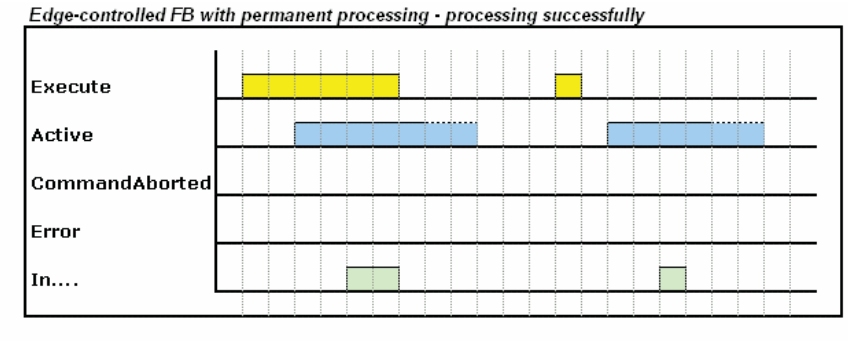
Fig. 8-14: I/Os of state-controlled FBs with permanent processing - processing was aborted

Inputs and outputs of edge-controlled function blocks with permanent processing

Control / Processing	I/O	Variable name	Description
edge-controlled / permanent	I	Execute	With a positive edge of Execute the input variables are included. New input values get effective not until a fresh positive edge of Execute The change of the edge at "Execute" is sufficient to start the module. For further processing the status of "Execute" is not relevant. With a fresh change of edge during processing the old task is discarded, the inputs are reassumed and the task is continued with the new values (re-trigger).
	O	In….	If "In…." is TRUE, the function block has reached its final position, but is operative further on to "keep" the achieved result and thus in in a durable final condition. Data outputs are valid now. As the FB remains operative, "Active" must also remain TRUE. This also applies, if "In…." is deactivated again. "Error" and "CommandAborted" are FALSE! If the control input "Execute" is FALSE at the termination of the task, "In…." remains TRUE for exactly one PLC cycle. If "Execute" is TRUE, "In…." remains TRUE till "Execute" is cleared.
	O	Active	If "Active" is TRUE, the function block works its real task. Potential pre-processings are not identified with this output! As the FB remains permanent operative, "Active" remains TRUE till the module is terminated with "Error" or "CommandAborted". As long as "Active" is TRUE, "Error" or "CommandAborted" must be FALSE!
	O	CommandAborded	If "CommandAborted" is TRUE, the function block was aborted and is in a final condition. "In….", "Active" and "Error" are FALSE! If the control input "Execute" is FALSE at the time of abortion, "CommandAborted" remains TRUE for exactly one PLC cycle. If "Execute" is TRUE, "CommandAborted" remains TRUE till "Execute" is cleared.
	O	Error	If "Error" is TRUE, the function block was aborted because of an error and is in a final condition. If the control input "Execute" is FALSE at occurring error, "Error" remains TRUE for exactly one PLC cycle. If "Execute" is TRUE, "Error" remains TRUE till "Execute" is cleared.

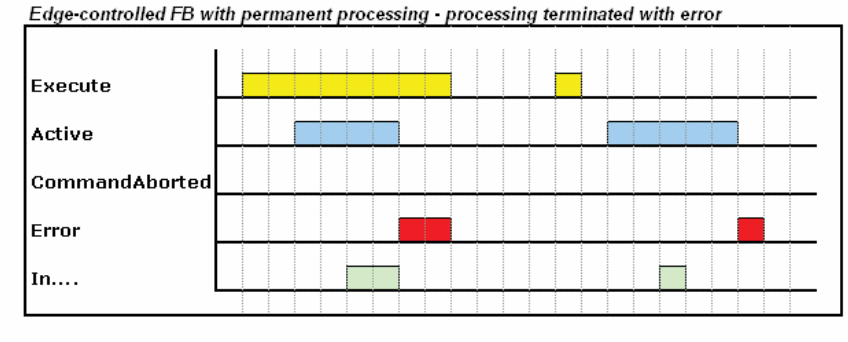
Fig. 8-15: I/O of edge-controlled FBs with terminating processing

Note: If it is necessary that special inputs are not only assumed with the edge at "Enable", but cyclically during module processing, it must be explicit documented and not identified with variable names!



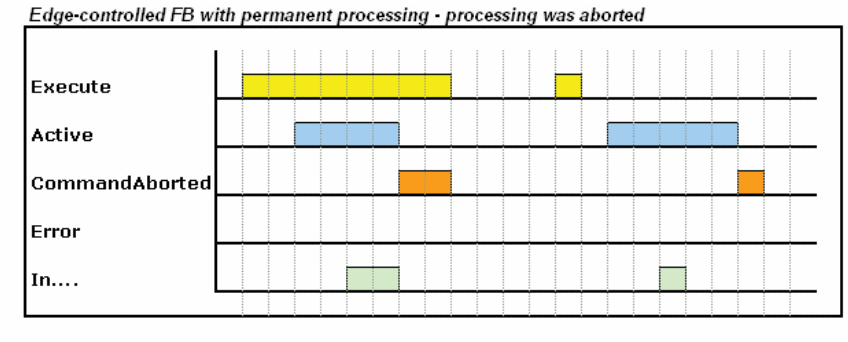
StateControlDiagram_FP1_EN.bmp

Fig. 8-16: I/Os of edge-controlled FBs with permanent processing - processing successfully



StateControlDiagram_FP2_EN.bmp

Fig. 8-17: I/Os of edge-controlled FBs with permanent processing - processing was aborted with error



StateControlDiagram_FP3_EN.bmp

Fig. 8-18: I/Os of edge-controlled FBs with permanent processing - processing was aborted

Sequence of the inputs and outputs

The interface at modules should be structured according to the following sequence:

Inputs

- Activation input (e.g., Execute, Enable...)
- Further inputs (optional)

Outputs

- Done (if available)
- Active (if available)
- InOperation (if available)
- CommandAborted (if available)
- Error
- ErrorID
- ErrorIdent
- Further outputs (optional)

Inputs/outputs

- Further inputs/outputs (optional)

Examples for state- and edge-controlled function blocks

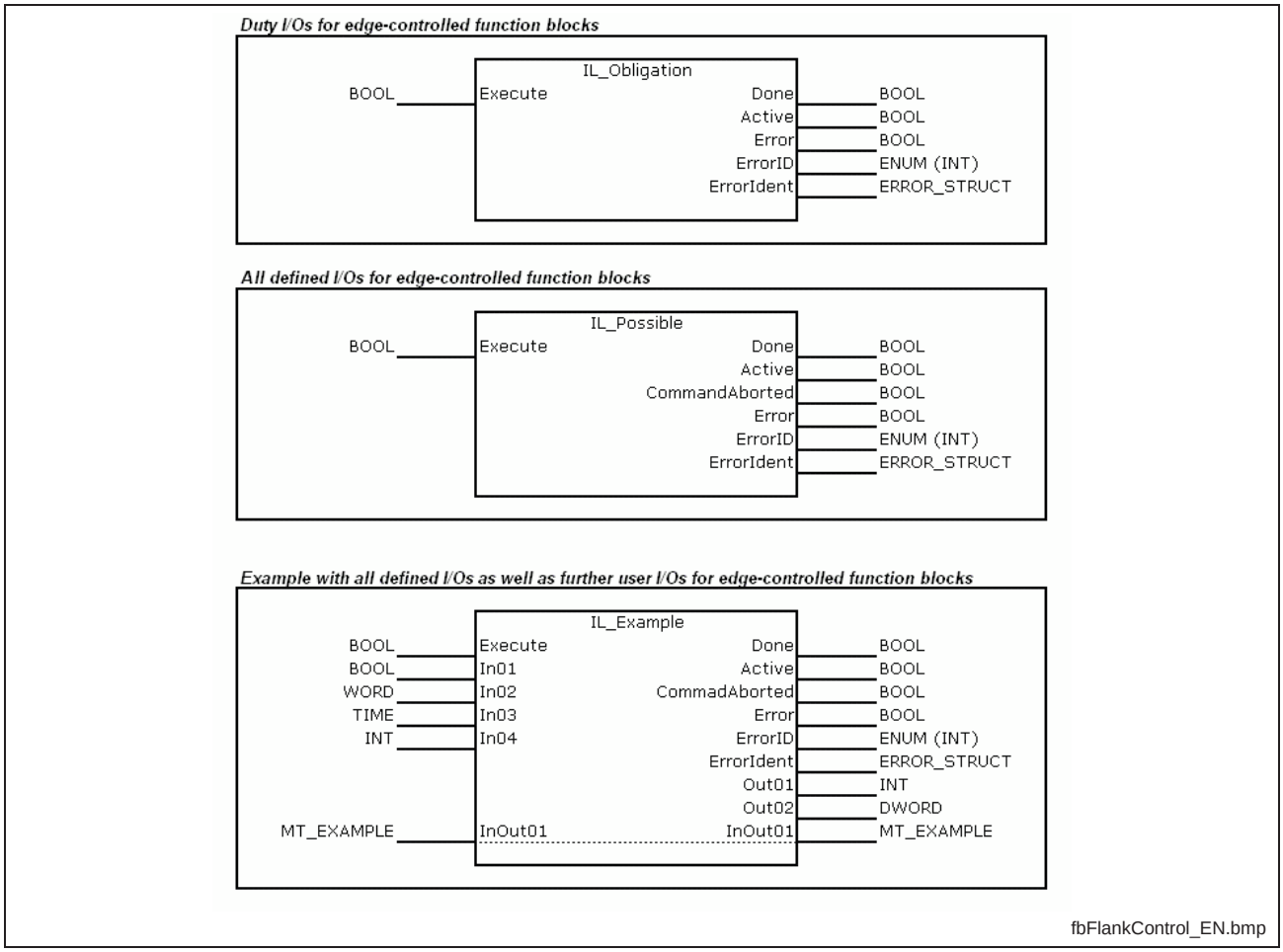


Fig. 8-19: I/Os for edge-controlled function blocks

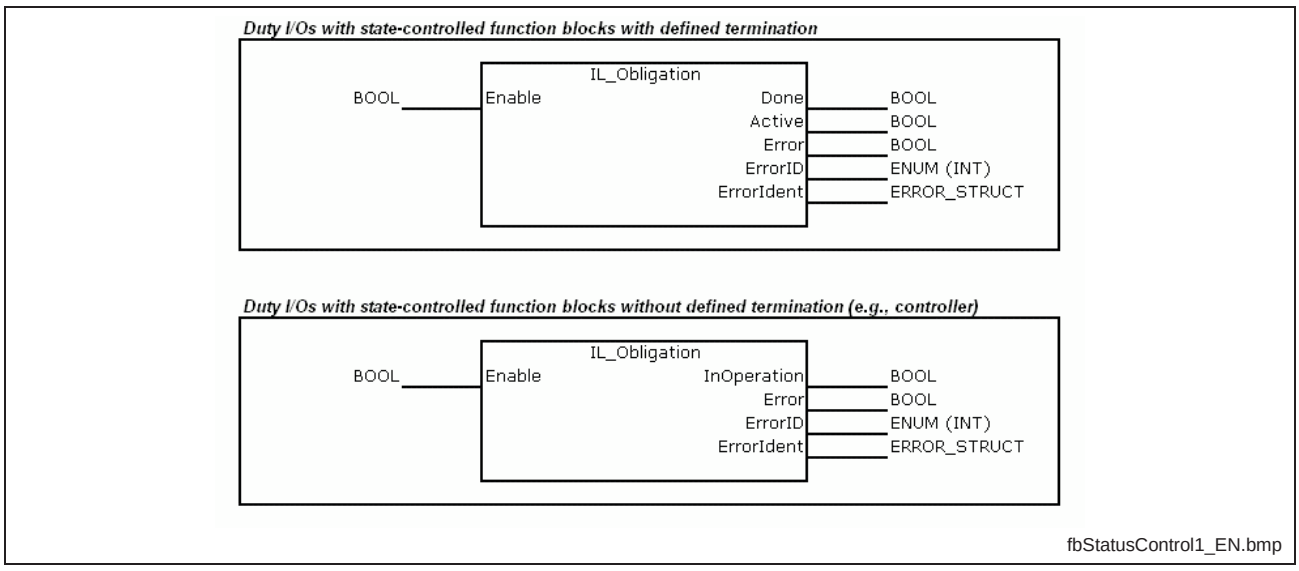


Fig. 8-20: I/Os for state-controlled function blocks

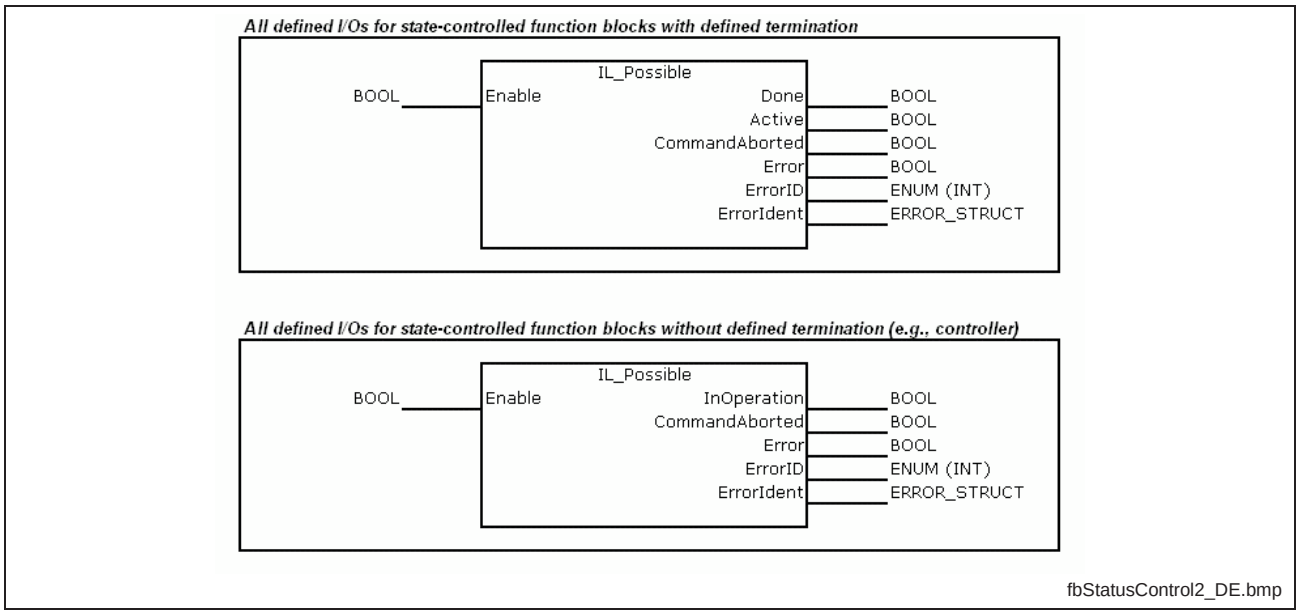


Fig. 8-21: I/Os for state-controlled function blocks

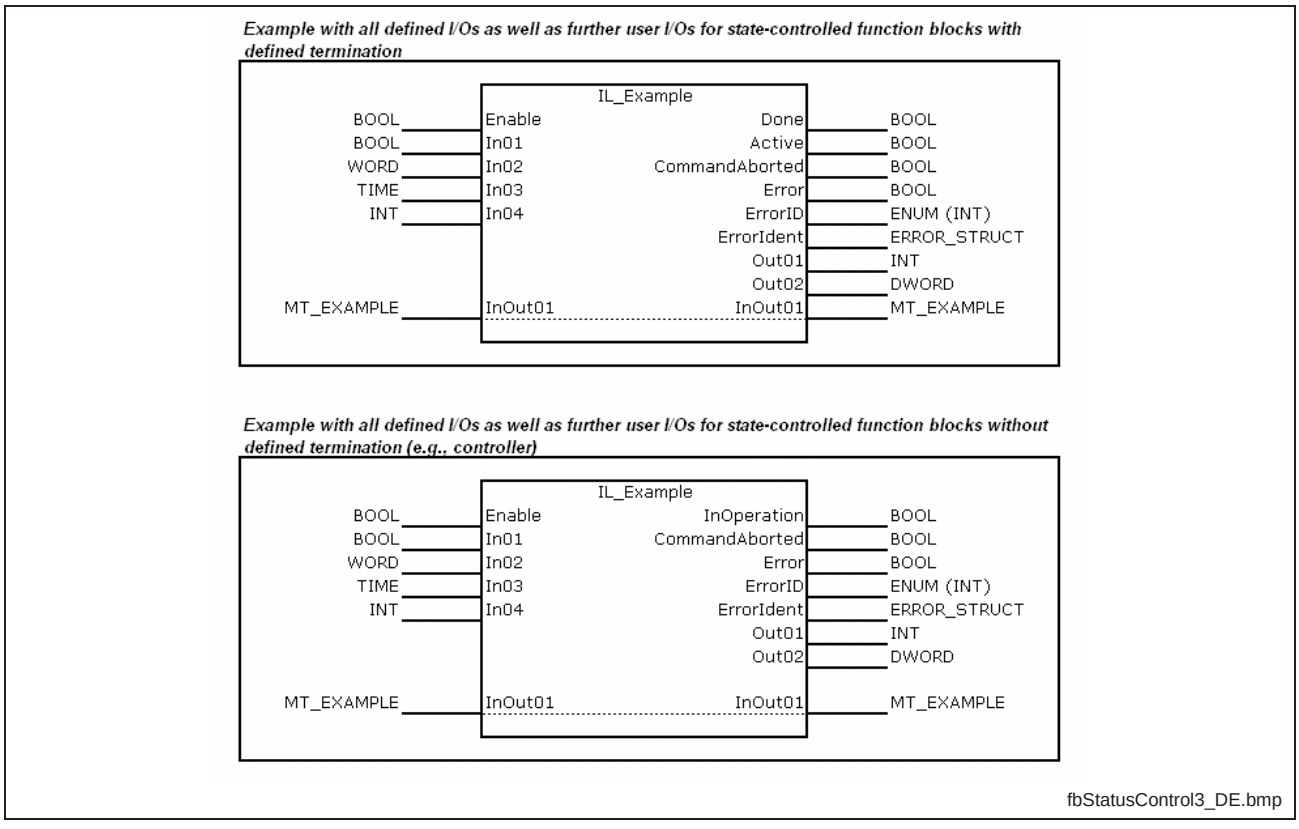


Fig. 8-22: I/Os for state-controlled function blocks

Further recommendations for the naming of identifiers

The naming of inputs and outputs should be as standardized as possible. The following table shows the input and outputs identifiers that are already in use and a short description of them.

Identifier (EN) written out	Identifier (EN) token	Description
Acceleration	Acc	acceleration
Acknowledge	Ack	acknowledge signal
Active		module is active (outputs are invalid)
Automatic	Auto	automatic
Actual	Act	actual
Axis	Ax	axis
Channel	Ch	channel
Clear	Clr	clear
Close		close
Communcation	Com	communication
Contact		contact
Continuous	Cont	continuous
Control	Ctrl	control
Counter		counter
Deceleration		deceleration
Decrement	Dec	decrement
DerivativeGain	Kd	derivative gain
Diameter		diameter
Distance	Dist	distance
Door		door
Drive	Drv	drive
Emergency		emergency
Fast		fast
Feedconstant		feed constant
Increment	Inc	increment
Incremental		incremental
IntegralGain	Ki	integral gain
Jerk		jerk
Jogging	Jog	jogging
JoggingMode	JogMode	jogging mode (continuous, incremental....)
Length		length
Lock		lock
Machine	Mach	machine
Manual	Man	manual
Maximum	Max	maximum
Middle		middle speed
Minimum	Min	minimum

ModuloValue		modulo value
Number	No	number
Off		off
On		on
Open		open
OperationMode	OpMode	operation mode (automatic, manual, ..)
ParameterNumber		SERCOS encoded parameter identical number
Pause		pause
Position	Pos	position
Power	Pow	power
Preset		preset
PresetValue		preset value
Probe		probe
Program	Prog	program
ProportionalGain	Kp	proportional gain
Rapid	Rap	rapid
Reduced	Red	reduced
Reset		reset
Slow		slow
Spindle	Sp	spindle
Standstill		standstill
Station		station
Status		status
Stop		stop
Supply	Sup	supply
Time		time
Unit		unit
Unlock		unlock
Value	Val	data value
Velocity	Vel	velocity
Winder		winder
Window		window (e.g., correction window)

Fig. 8-23: Overview of the available identifiers

9 Standardized error handling

Definition of use

Note: The use of standardized error handling is mandatory for all BRC functions and function blocks!

Motivation

A conclusive and universal concept of the error handling facilitates and shortens the troubleshooting. It is the prerequisite for the customer acceptance when changing within the systems of Bosch Rexroth.

Description

A good error handling offers a fast and simple diagnosis, but also the possibility to get detailed information about a problem. A granulation makes sense, as these requirements disagree in parts. So the error handling of Bosch Rexroth is divided into three diagnostic steps.

Step1 An error is available.

Step2 Rough information about the error.

Step3 Detailed information about the error.

Every function block with the error handling of Bosch Rexroth offers these three diagnostic steps. A simple boolean variable ("**Error**") displays an upcoming error. In the second diagnostic step an enumerator ("**ErrorID**") directly displays a rough allocation of the error. For detailed information a structure ("**ErrorIdent**") is available that can read out the source of the error as well as the exact error code. The detailed information about the error can be found in the corresponding documentation based on error source and error code.

Structure of "Error"

The type of "Error" is BOOL. The output is TRUE, if an error was detected. The behavior of "Error" depends on input "Execute" or "Enable" (see ???, ???).

Structure of "ErrorID"

The data type of "ErrorID" is ERROR_CODE (ENUM). This data type is provided in "RIL_CommonTypes.lib". "ErrorID" provides a standard rough error description on all IndraLogic systems. "ERROR_CODE" contains the following enumerators:

Enumerator	Code	Description
NONE_ERROR	16#0000	no error code available
INPUT_INVALID_ERROR	16#0001	invalid input allocation
COMMUNICATION_ERROR	16#0002	communication error
RESOURCE_ERROR	16#0003	source not available
ACCESS_ERROR	16#0004	faulty / incorrect access to data
STATE_MACHINE_ERROR	16#0005	state machine at invalid value
INPUT_RANGE_ERROR	16#0006	the value of one or several inputs is outside the defined limits
CALCULATION_ERROR	16#0007	calculation error
DEVICE_ERROR	16#0008	drive error
OTHER_ERROR	16#7FFE	undefined error (can not be allocated to another ID)
SYSTEM_ERROR	16#7FFF	system error

Fig. 9-1: "ErrorID" – ERROR_CODE (ENUM)

Structure of "ErrorIdent"

"ErrorIdent" is a structure that consists of three elements. The standard allocation of the error structure 0.

- ErrorTable (ERROR_TABLE)
- ErrorAdditional1 (DWORD)
- ErrorAdditional2 (DWORD)

ErrorTable ENUM (15Bit with reservations for systems, hardware and user area): identifies the "Error table", from where error numbers are entered to ErrorAdditional.

ErrorAdditional1 variable occupied depending on the error table, e.g., SERCOS error...

ErrorAdditional2 as additional error information, if necessary, depending on the ErrorTable

Enumerator	Code	Description
NO_TABLE_USED	16#0000	no table allocated
SERCOS_TABLE	16#0010	Sercos error (error at communication via Sercos)
MLD_TABLE	16#0020	drive error (error of PLC FBs of the drive PLC)
MLC_TABLE	16#0030	MLC error (error of PLC FBs of the MLC control)
MTX_TABLE	16#0040	MTX error (error of PLC FBs of the MTX control)
MLP_TABLE	16#0050	MLP error (error of PLC FBs of the PC-based control)
PLC_TABLE	16#0060	PLC error
INDRV_TABLE	16#0070	IndraDrive error (IndraDrive signals error via PLC FB)
DIAX_TABLE	16#0080	Diax error (Diax drive signals error via PLC FB)
ECO_TABLE	16#0090	EcoDrive error (EcoDrive signals error via PLC FB)
PB_DP_TABLE	16#0130	ProfibusDP error (ProfibusDP signals error via PLC FB)
DEVICENET_TABLE	16#0140	DeviceNet error (DeviceNet signals error via PLC FB)
ETHERNET_TABLE	16#0150	Ethernet error (Ethernet signals error via PLC FB)
ETHERNET_IP_TABLE	16#0151	EthernetIP error (EthernetIP signals error via PLC FB)
INTERBUS_TABLE	16#0160	Interbus error (Interbus signals error via PLC FB)
F_RELATED_TABLE	16#0170	System overlapping error messages (e.g., from technology FBs)
USER1_TABLE	16#1000	freely usable for the user
USER2_TABLE	16#1001	freely usable for the user
USER3_TABLE	16#1002	freely usable for the user
USER4_TABLE	16#1003	freely usable for the user
USER5_TABLE	16#1004	freely usable for the user
USER6_TABLE	16#1005	freely usable for the user
USER7_TABLE	16#1006	freely usable for the user
USER8_TABLE	16#1007	freely usable for the user
USER9_TABLE	16#1008	freely usable for the user
USER10_TABLE	16#1009	freely usable for the user

Fig. 9-2: "ErrorTable" (ENUM)

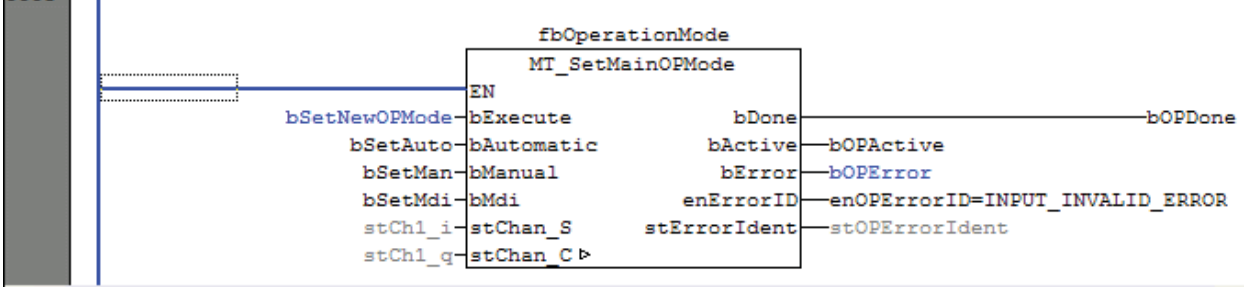
Examples of the error handling

```
0104 VAR CONSTANT
0105 (*System spezifische Fehlercodes*)
0106     MT_NO_ERROR_CODE            :DWORD:=16#00000000;    (*Kein Fehlercode verfügbar*)
0107     MT_INPUT_INVALID_ERROR      :DWORD:=16#00010001;    (*Falsche Eingangsbelegung*)
0108
0023 (*Eingangsbelegung ist ungültig*)
0024 ELSE
0025     enErrorID:=INPUT_INVALID_ERROR;                     (*Enum aus "ERROR_CODE" zuweisen*)
0026     stErrorIdent.Table:=MTX_TABLE;                      (*Enum aus "ERROR_TABLE" zuweisen*)
0027     stErrorIdent.Additional1:=MT_INPUT_INVALID_ERROR;   (*System spezifischen Fehlercode zuweisen*)
0028     stErrorIdent.Additional2:=MT_NO_ERROR_CODE;         (*System spezifischen Fehlercode zuweisen*)
```

FehlerhandlingProg.bmp

Fig. 9-3: Write error outputs in the FB

```
0035 fbOperationMode
0036     bSetNewOPMode := TRUE
0037     bSetAuto := FALSE
0038     bSetMan := FALSE
0039     bSetMdi := FALSE
0040     bOPDone := FALSE
0041     bOPActive := FALSE
0042     bOPError := TRUE
0043     enOPErrorID := INPUT_INVALID_ERROR
0044 stOPErrorIdent
0045     .Table := MTX_TABLE
0046     .Additional1 := 16#00010001
0047     .Additional2 := 16#00000000
```



FehlerhandlingOnline.bmp

Fig. 9-4: Interpretation of the error outputs in the implementation

MTX Fehlertabelle

Additional 1	Additional 2	Kurztext GER	Kurztext ENG
0x00000000	0x00000000	Kein Fehlercode verfügbar	no error code available
0x00010001	0x00000000	Eingänge sind falsch beschaltet	invalid input error

Fehlertabelle.bmp

Fig. 9-5: Example of an error table

10 Standardization of library names

Definition of use

Note: The following procedure about the specification of library names is mandatory for Bosch Rexroth and must be applied as described!

Motivation

A system overlapping standardized procedure at the specification of library identifiers simplifies the management and the allocation of the libraries.

Description

As the basis for the library names, the contents of a library should be summarized to a concise term with reference to the project and/or product. For whole BRC, token are specified for system-specific as well as for system-independent libraries (see Fig. 10-1). These one must be added to the defined term as prefix with underline. A version identification can be added as suffix separated with underline. The version identification consists of figures and may not contain any letters. Upper and lower case must be considered. The library identifiers should be created in English.

Prefix	System dependence	Allocation	Description	Example
RIL	no	-	Contains system-independent functions and function blocks that are not PLCopen and motion oriented, as well as corresponding data types	RIL_Utilities.lib
RIH	no	Hardware	Contains functions and function blocks without system reference (e.g., MLC, MTX, ...) that only run on the corresponding hardware (e.g., CML40, CMP60, ...)	RIH_CML40.lib
ML	yes	MLC	Contains MLC specific functions, function blocks and data types	ML_PLCOpen.lib
MP	yes	IndraLogic	Contains IndraLogic specific functions, function blocks and data types	MP_PLCOpen.lib
MS	yes	Synax	Contains Synax specific functions, function blocks and data types	MS_BaseSL13.lib
MSV	yes	Synax + VisualMotion	Contains functions, function blocks and data types for the Synax and VisualMotion systems	MSV_AcyclicComm.lib
MT	yes	MTX	Contains MTX specific functions, function blocks and data types	MT_MTX.lib
MV	yes	VisualMotion	Contains VisualMotion specific functions, function blocks and data types	MV_Motion.lib
MX	yes	Drive PLC	Contains functions, function blocks and data types especially for drive PLC	MX_BaseMPH02_01.lib

Fig. 10-1: Overview of the prefixes for libraries

Structure of the library name without version identification: "Prefix_Term.lib".

Structure of the library name with version identification: "Prefix_Term_VersionIdentification.lib".

Note: The use of a version identification is optional!

The version identification only consists of figures and may not contain letters!

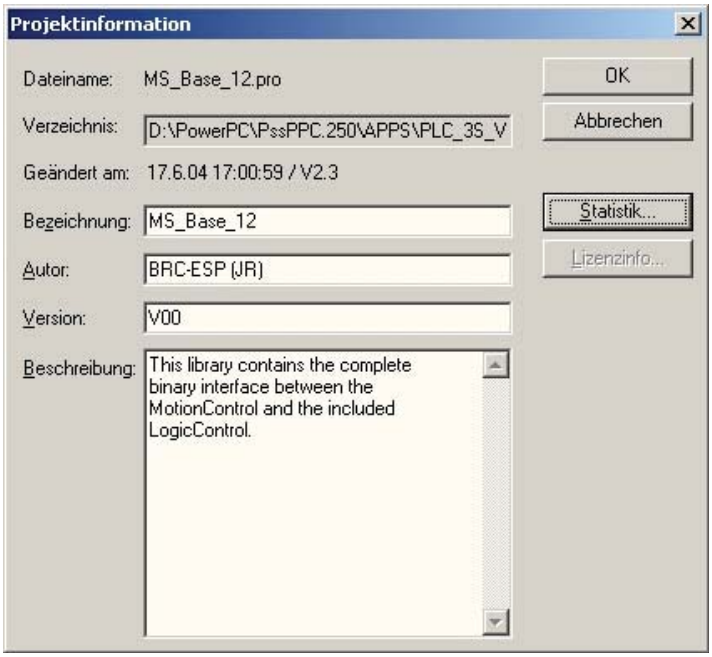
11 Standardization of the version management of libraries

Definition of use

Note: The use of the version function is optional! If a version function is used, the following procedure must be realized.

Motivation

In the current version, IndraLogic only provides limited possibilities of version management, target attachment and management of a history ("Project"-"Project information"). This limitation can be avoided in parts by using a version function.



ProjectInformation_DE.bmp

Fig. 11-1: Project information of a library in IndraLogic

Description

IndraLogic offers the possibility to create internal as well as external libraries. Internal libraries are completely programmed with programming language of IEC61131. External libraries are realized with a high level language (i.e. C/C++).

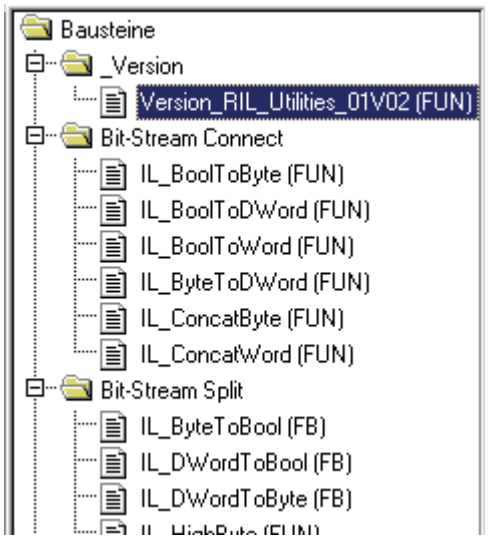
As the program code of a POE at an external library is linked via the POE designation at download, this can be used for the connection of the POE and thus of the library to a special firmware and thus to a special target.

The version management, the conditional target link and the management of a history can be provided in an aligned, simplified folder structure because of an aligned, simplified version function.

Folder and designation[1]

A function with designation `**Version**_*libraryname***Vyy**´ (Syntax A) or with designation **Version**_*libraryname*_**xxVyy**´ (Syntax B), if the library name does not contain the version number, is created in a folder `_Version´ in the root of the library module.

Note: The folder name of the version function should start with a leading underline to find it always at the same position.



RIL_Utilities3.bmp

L: Tab `Module´

Fig. 11-2: Structure of the library

Folder of the version function

_Version

Fig. 11-3: Folder of the version function

Designation of the version function (Syntax A)

Version _ LibrarynameVyy

Prefix: Version_
Suffix: Vyy

Fig. 11-4: Designation of the version function at a library name with version identification (Syntax A)

Designation of the version function (Syntax B)

Version _ Libraryname _ xxVyy

Prefix: Version_
Suffix: _xxVyy

Fig. 11-5: Designation of the version function at a library name without version identification (Syntax B)

[1] xx: Version number
yy: Release number

Designation of library	Syntax type	Prefix	Suffix	Designation of the library function
RIL_Utilities_02.lib	A	Version_	V05	Version_ RIL_Utilities_02V05
RIL_Utilities.lib	B	Version_	_02V05	Version_ RIL_Utilities_02V05

Fig. 11-6: General examples for the designation of the library function

Designation of library	Syntax type	Prefix	Suffix	Designation of the library function
MS_Base_12.lib	A	Version_	V02	Version_MS_Base_12V02
RIL_Utilities_01.lib	A	Version_	V00	Version_RIL_Utilities_01V00
MX_BaseMPH02_01.lib	A	Version_	V03	Version_MX_BaseMPH02_01V03
MT_MTX.lib	B	Version_	_01V07	Version_MT_MTX_01V07
RMC_PLCopen.lib	B	Version_	_02V00	Version_RMC_PLCopen_02V00

Fig. 11-7: Examples for the designation of the library function

Interface, code and return value

Interface variable

The version function is provided with the following interface variables.

```
VAR_INPUT
      Dummy: BOOL;
END_VAR

VAR
END_VAR
```

Fig. 11-8: Interface variables

Program code

The style of the program code must ensure a compiling without error and that the defined return value of the function is returned. Every further program code should be avoided.

```
Version_RIL_Utilities_01V02:= TRUE;
```

Fig. 11-9: Example for program code of a version function

Return type

The return type of the version function is BOOL.

Return value

The return value of the version function is always TRUE.

Version management and history

The version management of the library is ensured by the project information (Fig. 11-1) and by the designation of the version function as well as by the standardized header of the version function that is described in this guideline.

The standardized header offers extensive information about purpose of use, preliminary handling, etc. because of the detailedness. The project information integrated in IndraLogic can not offer this, because of the window size and the missing sections.

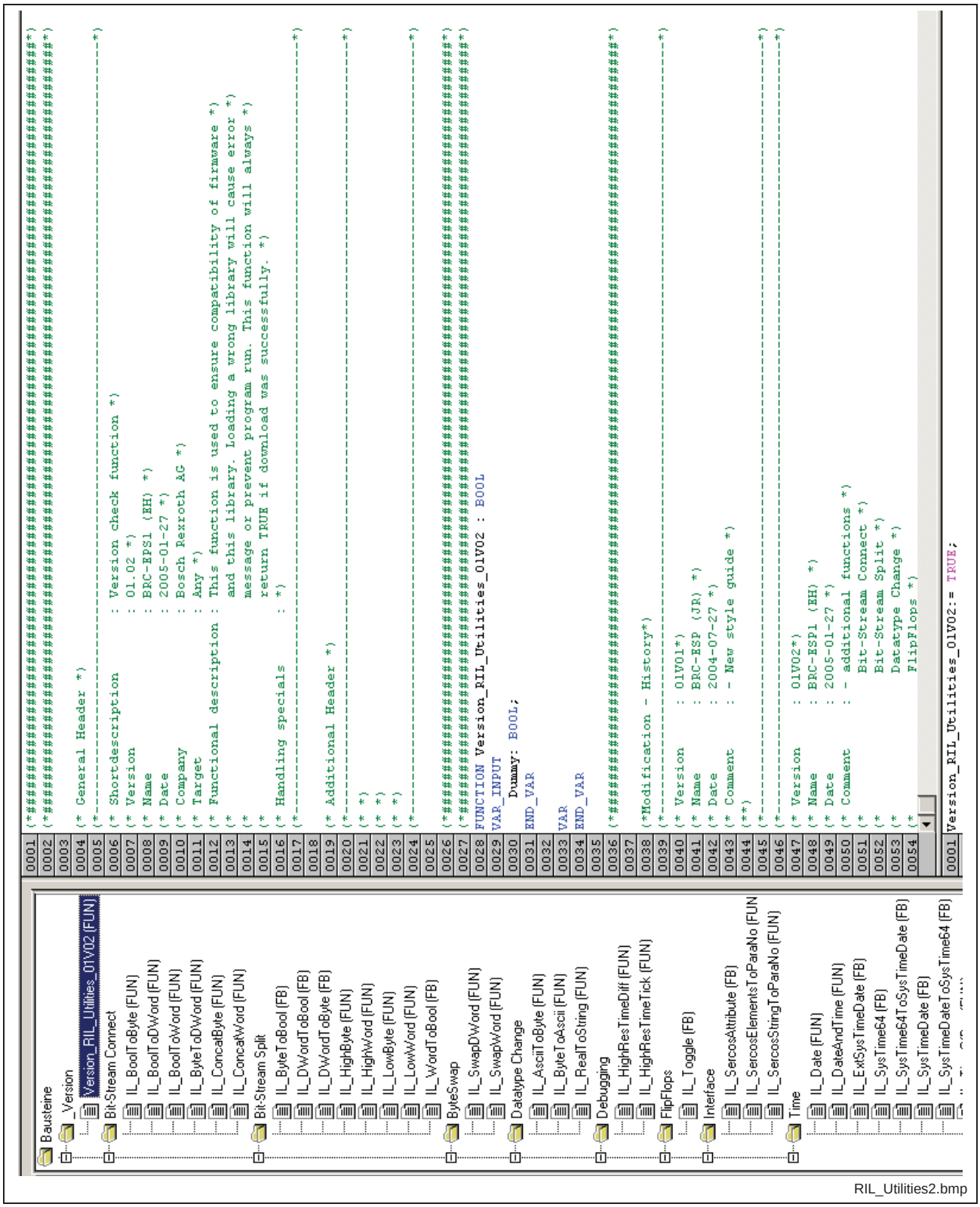
Allocation of the specification in the header

The specification in the header completely corresponds to the library itself, but not to the version function itself.

To document the whole development of the library, it is useful to add the record of revision as history in the declaration of the version function. The structure of the history is equivalent to the description of the record of revision that is described in this guideline.

Allocation of the specification in the history

The specification in the history corresponds to the library itself, but not to the version function itself.



RIL_Utilities2.bmp

L: Deceleration and implementation (ST)

Fig. 11-10: Example of an implementation of a version function

Target link (version protection)

The version function is only possible at external libraries due to principle. The version function is provided with an entry in the functional table `CstExtRefTable[]´ of `RtsCst.c´ with the same name as well as a corresponding runtime function (Fig. 11-11) like any other POE that is implemented in the firmware.

```
static ExtRef CstExtRefTable[] =
{
  /* {Function-name (max. 31 characters), Function-pointer}                    */
  /* ************************************************************************ */
  /* ************************************************************************ */
  /* ************************************************************************ */
  /* COMMON POU - version check of libraries - alphabetic order               */
  /*<--------31 CHAR-!!!---------->*//* Function-name (max. 31 characters)   */
  {"Version_CommonTypes_01V00", (void (*)(void))iboMB_Version_CommonTypes_01V00},
  {"Version_Diagnosis_01V00", (void (*)(void))iboMB_Version_Diagnosis_01V00},
  {"Version_Standard_01V00", (void (*)(void))iboMB_Version_Standard_01V00},
  /* ************************************************************************ */
  /* SYNAX AND VM POU - version check of libraries - alphabetic order         */
  /*<--------31 CHAR-!!!---------->*//* Function-name (max. 31 characters)   */
  {"Version_AcyclicComm_MSV01V00", (void (*)(void))iboMS_Version_AcyclicComm_MSV01V00},
  {"Version_AcyclicComm_MSV01V01", (void (*)(void))iboMS_Version_AcyclicComm_MSV01V01},
  {"Version_Utilities_01V00", (void (*)(void))iboMS_Version_Utilities_MSV01V00},
  /* ************************************************************************ */
  /* SYNAX POU - version check of libraries inside *.obj - alphabetic order   */
  /*<--------31 CHAR-!!!---------->*//* Function-name (max. 31 characters)   */
  /*                                                                          */
```

FunctionTable_DE.bmp

L: `RtsCst.c´

Fig. 11-11: Functional table

Message during download

If a firmware library is included to a user project, all modules contained in the firmware library are included during a download[2]. If one or several functions can not be included, the download is canceled with a list of the missing functions (Fig. 11-12).

ErrorOnVersionCheck_DE.bmp

Fig. 11-12: Message of a failed version check

With this message the user recognizes, which function from which library could not be connected and the library version the user has tried to use. With a reference list (e.g., release notes) the user has the possibility to determine which libraries in which release are available at his firmware.

[2] The POE link and thus the version protection possibly can not be assumed on all targets.

The concept of the version management enables a **selective** downward compatibility, i.e. older releases of the libraries run on a newer firmware platform further on, if the originator of the library leaves the respective "old" version functions in the function table `CstExtRefTable[]´ of the `RtsCst.c´ (Fig. 11-13).

```
static ExtRef CstExtRefTable[] =
{
  /* {Function-name (max. 31 characters), Function-pointer}                    */
  /* ************************************************************************* */
  /* ************************************************************************* */
  /* ************************************************************************* */
  /* COMMON POU - version check of libraries - alphabetic order                */
  /*<--------31 CHAR-!!!---------->*//* Function-name (max. 31 characters)    */
  {"Version_CommonTypes_01V00", (void (*)(void))iboMB_Version_CommonTypes_01V00},
  {"Version_Diagnosis_01V00", (void (*)(void))iboMB_Version_Diagnosis_01V00},
  {"Version_Standard_01V00", (void (*)(void))iboMB_Version_Standard_01V00},
  /* ************************************************************************* */
  /* SYNAX AND VM POU - version check of libraries - alphabetic order          */
  /*<--------31 CHAR-!!!---------->*//* Function-name (max. 31 characters)    */
  {"Version_AcyclicComm_MSV01V00", (void (*)(void))iboMS_Version_AcyclicComm_MSV01V00},
  {"Version_AcyclicComm_MSV01V01", (void (*)(void))iboMS_Version_AcyclicComm_MSV01V01},
  {"Version_Utilities_01V00", (void (*)(void))iboMS_Version_Utilities_MSV01V00},
  /* ************************************************************************* */
  /* SYNAX POU - version check of libraries inside *.obj - alphabetic order   */
  /*<--------31 CHAR-!!!---------->*//* Function-name (max. 31 characters)    */
  /*                                                                           */
```

FunctionTable2_DE.bmp

L: `RtsCst.c´

Fig. 11-13: Functional table

Summary version protection

The version function limits the download of the whole project at the integration of the library to the valid system as well as to the valid release as the corresponding system function is not available at invalid systems or invalid releases and therefore can not be addressed.

12 Check list for libraries and FBs

The following check lists support the control of libraries and function blocks. The documented properties must be fulfilled so that the libraries and the modules correspond with the StyleGuide.

Libraries

Criterion	Reference property	Example
Font	- CourierNew, 8	
Name of library	- English - Prefix (RIL_, RIH_, ML_, ...) - Upper and lower case without underline	ML_PLCOpen.lib RIL_Utilities.lib
Structure of the folder	- English	
Version function	- English - In folder "_Version" - Name (Version_Library name_xxVyy) - Header (like FB) - History (like FB) - Return value always TRUE	 Bsp.-Versionsfunktion.bmp

Abb. 12-1: Check list of libraries

Function blocks

Criterion	Reference property	Example
Name of module	- English - Prefix (MC_, IH_, ML_...) - Upper and lower case without underline	MC_MoveAbsolute (FB)
Header	- English - Structure according to StyleGuide	 Bsp.-Header.bmp
History	- English - Structure according to StyleGuide	 Bsp.-History.bmp
Identifier data type	- English - Prefix (ML_, MX_, MP_...) - everything upper case with underlines	MC_START_MODE (ENUM)
Identifier variables	- English - Prefix (optional – see StyleGuide) - Upper and lower case without underline - Suffix (optional – see StyleGuide)	fbMoveAxis1 : MC_MoveAbsolute; stDeviceCom : MX_COM_DATA; udiTelegram : UDINT; bAxTravelLim_i AT%IX1.0 : BOOL;
Behavior of the standard interfaces	- If standardized identifiers are used, the behavior must necessarily agree with the behavior described in the StyleGuide ! If another behavior is required, other identifiers must be used! Standardized identifiers are: - Enable	

	- Execute - ExecuteLock - Done - Active - InOperation - CommandAborted - Error - ErrorID - ErrorIdent	
	The sequence of the inputs and the outputs must be kept (see "Sequence of the inputs and outputs")	

Fig. 12-2: Checklist of function blocks